Turms specification

v0.1.0
Gravitalia

2025-10-24

Contents

Terminology

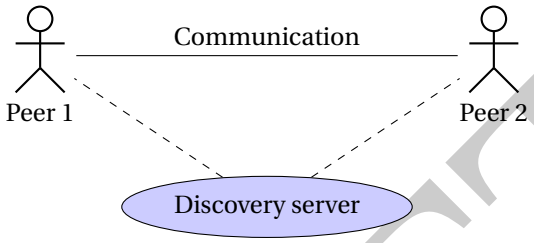
1. MUST This word, or the terms "REQUIRED" or "SHALL", mean that the definition is an absolute requirement of the specification.
2. MUST NOT This phrase, or the phrase "SHALL NOT", mean that the definition is an absolute prohibition of the specification.
3. SHOULD This word, or the adjective "RECOMMENDED", mean that there may exist valid reasons in particular circumstances to ignore a particular item, but the full implications must be understood and carefully weighed before choosing a different course.
4. SHOULD NOT This phrase, or the phrase "NOT RECOMMENDED" mean that there may exist valid reasons in particular circumstances when the particular behavior is acceptable or even useful, but the full implications should be understood and the case carefully weighed before implementing any behavior described with this label.
5. MAY This word, or the adjective "OPTIONAL", mean that an item is truly optional. One vendor may choose to include the item because a particular marketplace requires it or because the vendor feels that it enhances the product while another vendor may omit the same item. An implementation which does not include a particular option MUST be prepared to interoperate with another implementation which does include the option, though perhaps with reduced functionality. In the same vein an implementation which does include a particular option MUST be prepared to interoperate with another implementation which does not include the option (except, of course, for the feature the option provides.)

(Bradner, S. *Key words for use in RFCs to Indicate Requirement Levels.* RFC 2119. Mar. 1997. DOI: 10.17487/RFC2119)

These words are to be interpreted as described in BCP 14 [1] when they appear in all capitals. We will use “P2P” to mean “point-to-point, more generally peer-to-peer”.

1 Introduction

Gravitalia Turms ("Turms") is a point-to-point messaging service. Message exchanges are encrypted using multiple layers ("multi-layer encryption," "E2EE"). Message storage is encrypted at rest. Turms is not yet a peer-to-peer messaging service, as it does not provide indirect benefits to other peers[3].



1.1 Architecture

Direct communication between clients is often impossible, mainly due to firewalls and NAT. Turms' architecture takes these issues into account.

1.1.1 Peer discovery

There are several ways to find peers. Turms implements only two of them:

1. SDP sharing
2. Discovery server

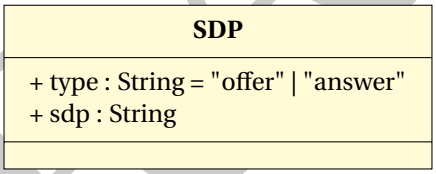
SDP sharing

Session Description Protocol (SDP) is an essential component of WebRTC. It allows two peers to connect directly to each other without going through a discovery server. SDP contains the codec, source address, and timing information for audio and video. During the initial initialization, SDP does not contain audio and video codecs. These are renegotiated on the fly when the two peers wish to communicate via audio or video. This choice avoids the slight overhead of finding the hardware beforehand and then during transmission, when it turns out to be unnecessary. SDP also contains information for initiating a secure connection (**??**). SDP sharing implies user shares its SDP outside Turms network.

Turms SDP MUST contains:

Name	Content	Description
v	0	SDP version.
o		Session origin.
s	-	Session name.
t	0 0	Session time.
a=fingerprint		DTLS fingerprint.
a=group	BUNDLE 0	Bundled media.
a=extmap-allow-mixed		Mixed RTP extensions.
m		Media description.
c		Connection info.
a=setup	actpass	DTLS role.
a=mid	0	Media ID.
a=sendrecv		Bidirectional flow.
a=sctp-port	5000	SCTP port.
a=ice-ufrag		ICE username.
a=ice-pwd		ICE password.
a=candidate	(multiple)	ICE candidates.
typ host		Local candidate.
typ srflx		Reflexive candidate.
raddr / rport		Related address/port.
a=end-of-candidates		End of ICE list.

The SDP MUST be passed in JSON format, as:



1.2 Goals

This point-to-point architecture choice ensures the resilience and privacy of telecommunications. However, authenticity is more difficult to preserve.

1.2.1 Privacy

The term “privacy” is semantically rich. It ranges from an individual’s physical condition to their most fundamental rights [4]. P2P connections avoid some threats. For instance, surveillance is made more tricky; uniquely if traffic is encrypted. Since data is stored locally, it relies on hardware security measures and only compromises one user in the event of a leak. However, it discloses the IP address, this gives an approximate geolocation of the peer. To maximize benefits, you SHOULD use onions, but it remains complicated.

1.2.2 Authenticity

Authenticity allows the author of a message to be proven. In P2P, authenticity is hard to prove. Turms allows individuals to authenticate themselves through an Autha instance, or to remain as “guests.”

1. **Autha instance with discovery server**: discovery server validates the decentralized identity, then generates a JWT. During signaling, the discovery server acts as a trusted third party. User

acquires the ID “user_id@discovery_server.tld.” Once the initial connection has been established, messages are authenticated by their signature.

2. **Autha instance without discovery server**: user generates a token that will be sent to the peer, who will verify it using the public key provided by the user's Autha instance.
3. **As guest**: untrustworthy and inauthentic by nature. Trust is acquired through the SDP transmission channel.

1.2.3 Integrity

Integrity ensures that messages has not been altered during transit. This component goes hand in hand with data encryption.

1.2.4 Resilience

P2P simplifies resilience. By design, there is no longer a single point of failure, except for the network itself.

2 Authentification

Authentication ensures that the message is encrypted and that it has not been tampered with. Turms relies on a combination of key establishment and transport layer security protocols, including DTLS for secure connection and X3DH/Double Ratchet for establishing and rotating end-to-end encryption (E2EE) keys. Turms MUST implement these three layers.

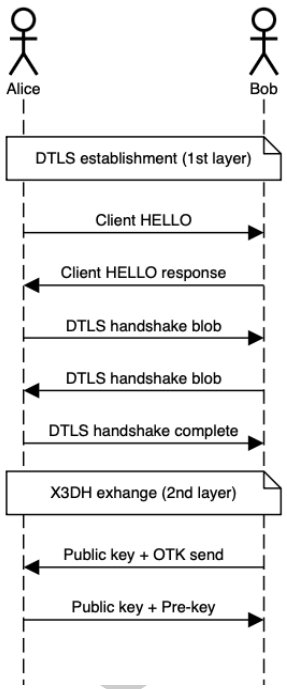


Figure 2.1: Diagram of exchanges before the the first message.

2.1 Datagram Transport Layer Security

Datagram Transport Layer Security (DTLS) is a protocol based on TLS, designed to prevent eavesdropping, tampering, and message forgery [5]. It guarantees a reliable and in-order delivery of data, unlike datagram transport. DLTS is the first security layer of the Turms protocol. You MUST implement a first security layer, ensuring encryption, authenticity and integrity.

See: RFC 9147.

2.2 Extended Triple Diffie-Hellman

In P2P, there is no trusted server to guarantee that Alice and Bob's public keys are correct. The security layer provided by DTLS ensures that the keys have not been altered by a third party (MITM).

If Alice initiated the WebRTC connection, then Bob initiates the key exchange. Bob, as initiator, send to Alice its public key and one-time key (IK_B, OPK_B).

$$
\begin{aligned}
DH_1 &= DH(IK_B, SPK_A) \\
DH_2 &= DH(EK_B, IK_A) \\
DH_3 &= DH(EK_B, SPK_A) \\
DH_4 &= DH(EK_B, OPK_A) \\
SK &= \mathrm{KDF}_{\text{Concat}}\left(DH_1 \,\|\, DH_2 \,\|\, DH_3 \,\|\, DH_4\right)
\end{aligned}
$$

with KDF a key derivation function, often based on HKDF with SHA-256.

$$
\begin{aligned}
(\text{RootKey}, \text{ChainKey}) &= \mathrm{KDF}_{\text{Root}}(SK) \\
\text{MessageKey} &= \mathrm{KDF}_{\text{Chain}}(\text{ChainKey})
\end{aligned}
$$

Bob checks Alice's pre-signed SPK_A is authentic:

$$\text{VerifySig}(IK_A, \text{Sig}_{IK_A}(SPK_A)) = 1$$

As described on Signal's specification, "the parties may compare their identity public keys IK_A and IK_B through some authenticated channel"[7, p. 7]. Turms has chosen to ensure P2P security by conducting exchanges over the secure DTLS channel. You MUST implement DTLS connection before exchange public keys. This choice mitigate subsections 4.1, 4.7 and 4.8 of X3DH protocol "Security considerations"[7], but creates possible non-deniability of communication between peers.

2.3 Double Ratchet

The Double Ratchet protocol extends X3DH by ensuring "Perfect Forward Secrecy" and "Post-Compromise Security"[6]. It combines two key update mechanisms:

- one Diffie-Hellman key ratchet (asymmetric);
- un symmetrical ratchet (based on a KDF).

After the X3DH exchange, both parties have the same initial root key:

$$RK_0 = \mathrm{KDF}_{\text{Root}}(SK_{\text{X3DH}})$$

Each party also has a pair of initial Diffie-Hellman keys:

$$(DH_s^{(0)}, DH_r^{(0)})$$

Each time a new public key $DH_r^{(t)}$ is received from the other party:

$$
\begin{aligned}
DH_{\text{out}}^{(t)} &= DH(DH_s^{(t-1)}, DH_r^{(t)}) \\
RK_t, CK_r^{(t)} &= \mathrm{KDF}_{\text{Root}}(RK_{t-1}, DH_{\text{out}}^{(t)}) \\
DH_s^{(t)} &\leftarrow \text{GenerateKeyPair()} \\
RK_t, CK_s^{(t)} &= \mathrm{KDF}_{\text{Root}}(RK_t, DH(DH_s^{(t)}, DH_r^{(t)}))
\end{aligned}
$$

Thus, with each Diffie-Hellman jump, the root key and the two strings (send/receive) are renewed. For each message sent, the send string is derived:

$$CK_s^{(i+1)}, MK_s^{(i)} = \mathrm{KDF}_{\text{Chain}}(CK_s^{(i)})$$

For each message received:

$$CK_r^{(j+1)}, MK_r^{(j)} = \text{KDF}_{\text{Chain}}(CK_r^{(j)})$$

$$\text{Encrypt}(MK_s^{(i)}, \text{plaintext}) \rightarrow \text{ciphertext}$$
$$\text{Decrypt}(MK_r^{(j)}, \text{ciphertext}) \rightarrow \text{plaintext}$$

A message key MUST NEVER be reused, to ensure perfect confidentiality of old messages even in the event of future compromise.

$$\begin{aligned}
RK_0 &= \text{KDF}_{\text{Root}}(SK_{\text{X3DH}}) \\
RK_t, CK_r^{(t)} &= \text{KDF}_{\text{Root}}\big(RK_{t-1}, DH(DH_s^{(t-1)}, DH_r^{(t)})\big) \\
RK_t, CK_s^{(t)} &= \text{KDF}_{\text{Root}}\big(RK_t, DH(DH_s^{(t)}, DH_r^{(t)})\big) \\
CK_x^{(i+1)}, MK_x^{(i)} &= \text{KDF}_{\text{Chain}}(CK_x^{(i)}), \quad x \in \{s, r\}
\end{aligned}$$

- **Perfect Forward Secrecy**: compromising a future key does not reveal previous messages.
- **Post-Compromise Security**: after a DH jump, the new keys become secure again.
- **Unidirectional derivation**: no derived key can be traced back to the root key.

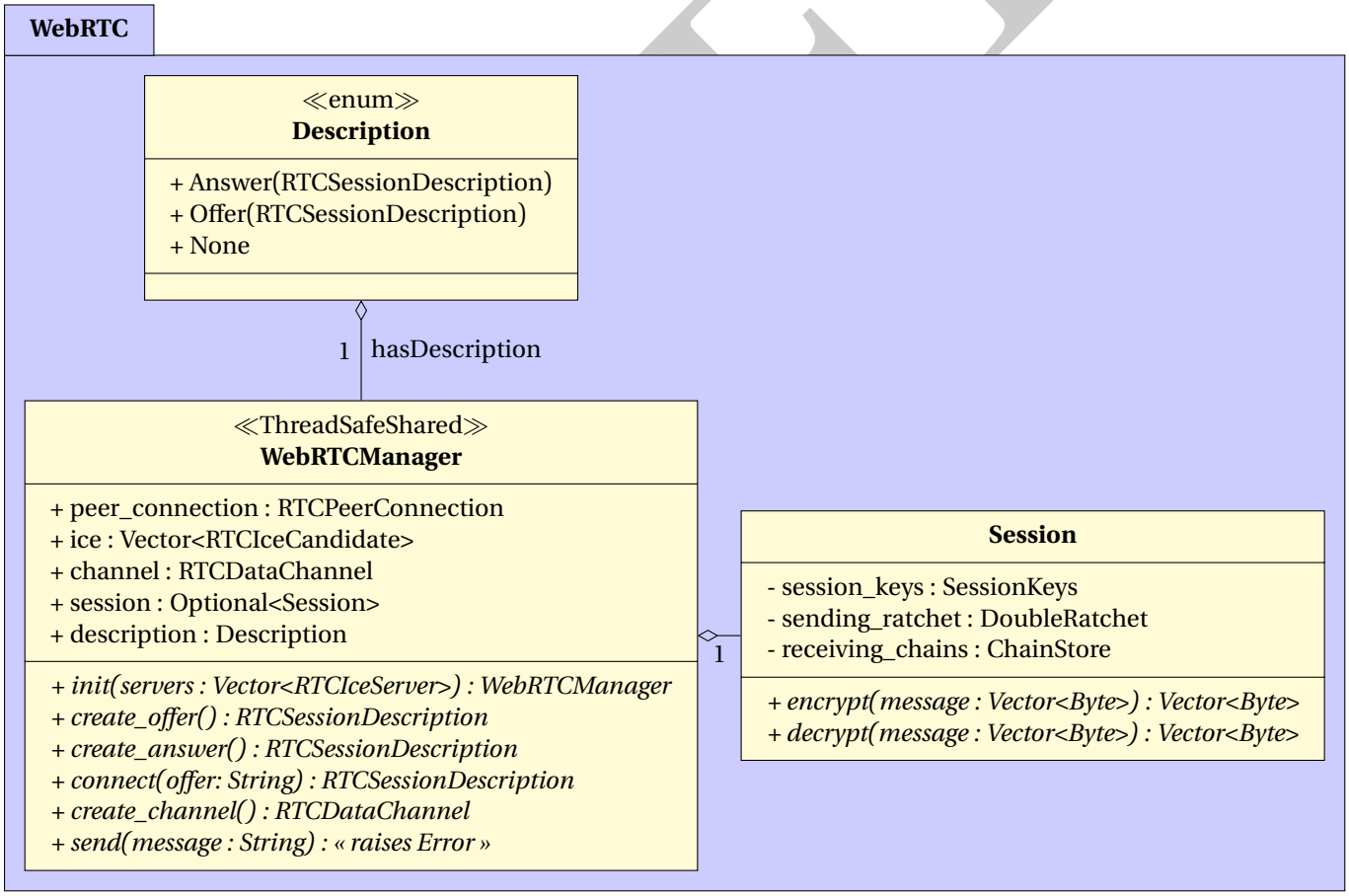
3 Messages

Messages are the standardized ways of communication between peers or servers. Implementation MUST be identical.

3.1 Network

Everything that passes through MUST be converted into bytes to ensure interoperability between unencrypted and encrypted messages. No strings over channel. Message serialization format MUST be JSON. Deserialization is performed according to the events below.

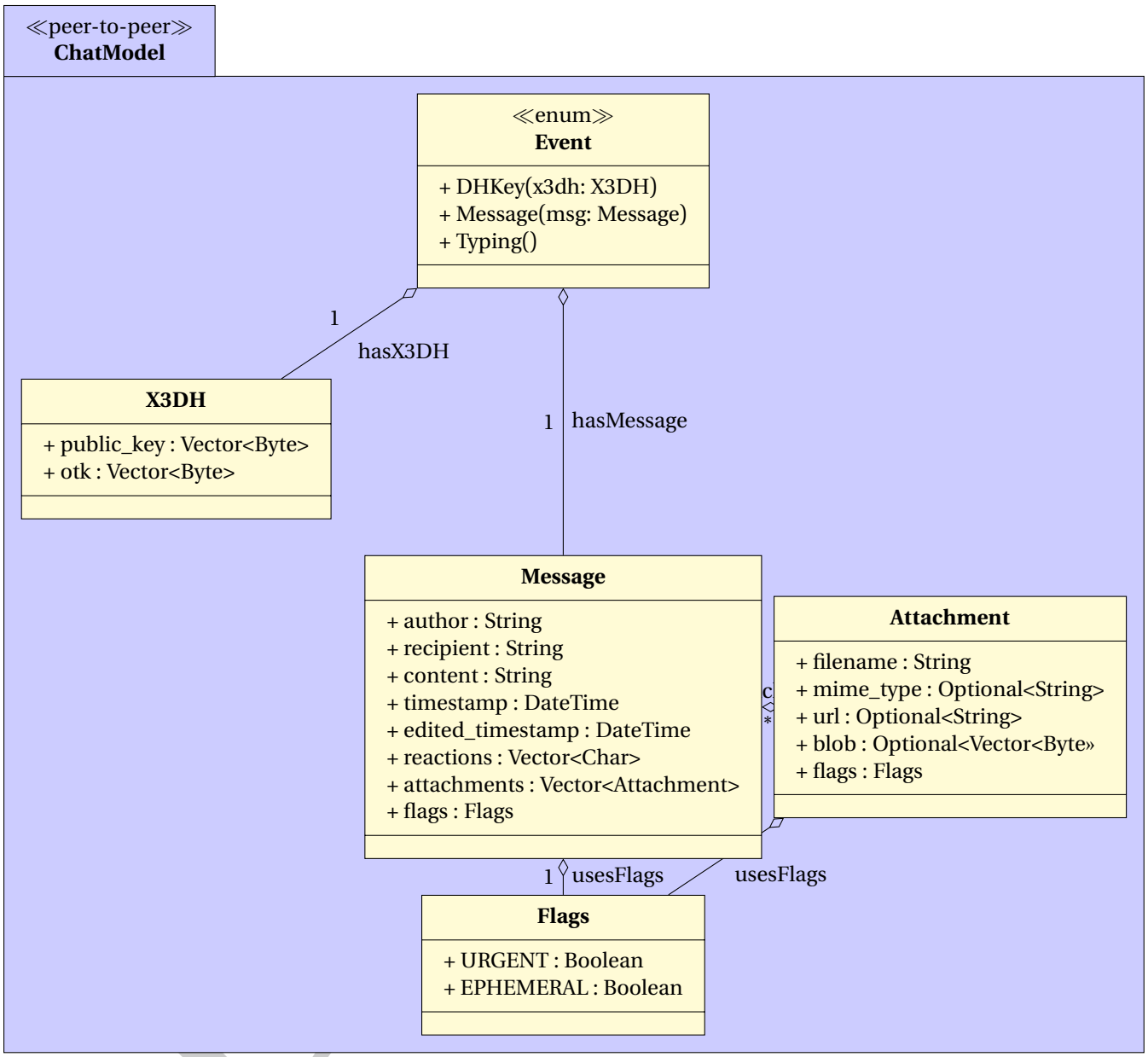
3.1.1 Transport



One, and only one, `WebRTCManager` MUST be used for one, and only one, WebRTC session. Once the WebRTC session has been created, after the exchange of offer, response to offer, and response to response to offer, both peers MUST exchange their public keys, OTK, and pre-keys (X3DH). Then, `Session` is created.

Transport MUST operate over Datagram Transport Layer Security.

3.2 Events



`Event` MAY be encrypted, depending if session is initialized. When `Event` is `Message(Message)` or `Typing()`, `Event` MUST be encrypted.

Bibliography

[1] Leiba, B. *Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words*. RFC 8174. May 2017. DOI: 10.17487/RFC8174.

[2] Bradner, S. *Key words for use in RFCs to Indicate Requirement Levels*. RFC 2119. Mar. 1997. DOI: 10.17487/RFC2119.

[3] G. Camarillo, Ed. *Peer-to-Peer (P2P) Architecture: Definition, Taxonomies, Examples, and Applicability*. RFC 5694. Nov. 2009. DOI: 10.17487/RFC5694.

[4] Cooper, A. and Tschofenig, H. and Aboba, B. and Peterson, J. and Morris, J. and Hansen, M. and Smith, R. *Privacy Considerations for Internet Protocols*. RFC 6973. July 2013. DOI: 10.17487/RFC6973.

[5] Eric Rescorla, Hannes Tschofenig, and Nagendra Modadugu. *The Datagram Transport Layer Security (DTLS) Protocol Version 1.3*. RFC 9147. Apr. 2022. DOI: 10.17487/RFC9147.

[6] Trevor Perrin and Moxie Marlinspike. *The Double Ratchet Algorithm*. Specification. Version 3. Sept. 2025. URL: https://signal.org/docs/specifications/doubleratchet/doubleratchet.pdf.

[7] Moxie Marlinspike and Trevor Perrin. *The X3DH Key Agreement Protocol*. Specification. Signal Messenger, Nov. 2016. URL: https://signal.org/docs/specifications/x3dh/x3dh.pdf.

Démonstration de la résilience d'un réseau P2P

A. CN

Gravitalia
acn@gravitalia.com

Mots-clés : Résilience · Pair-à-pair · Théorie des graphes · Théorie de la percolation

Résumé Cette démonstration, par la théorie des graphes, permet de comparer la résilience des réseaux pairs-à-pairs face aux réseaux centralisés, en partant de certains postulats empiriques.

1 Introduction

La démonstration modélise un graphe aléatoire de sommets (nœuds) éventuellement connectés entre eux. Chaque sommet représente un pair. Cette démonstration modélise un réseau pair-à-pair, où chaque pair agit au bénéfice des autres ; et non un réseau point-à-point. Nous considérons qu'il n'existe ni congestion réseau ni attaque sur le réseau.

2 Démonstration

Soit une famille de variables de Bernoulli indépendantes $(X_{i,j})_{1\leq i<j\leq n}$ de paramètre $p \in [0,1]$, où $n \in \mathbb{N}$ est le nombre de sommets, définissant un graphe d'Erdös-Rényi[3] $G(n,p)$ tel que $X_{i,j} = 1$ si l'arête (i,j) existe et 0 sinon.

Théorème 1 (Connexité). *$p(n) = \frac{\log n + c_n}{n}$ est seuil critique pour la connexité du graphe d'Erds-Rényi. Plus précisément,*

$$\mathbb{P}(G(n,p(n)) \textit{ est connexe}) \underset{c_n \to \pm\infty}{\to} \begin{cases} 1 \textit{ si } c_n \to +\infty \\ 0 \textit{ si } c_n \to -\infty \end{cases}$$

Lemme 1 (Composante géante). *Posons $p = \frac{c}{n}$, avec $c = n \cdot p$ la densité moyenne de connexions par sommet.*

- *Si $c < 1$, alors en moyenne chaque sommet à moins d'un voisin. Les composantes sont de tailles $\Theta(\log n)$.*
- *Si $c = 1$, alors il y a une transition du seuil de percoation. La plus grande composante est de taille $\Theta(n^{2/3})$.*
- *Si $c > 1$, alors une composante géante apparaît et contient une fraction significative des nœuds.*

A partir du lemme 1, on peut déterminer lorsqu'un réseau pair-à-pair est viable, et s'assurer que les données arrivent au pair voulu. Seuls α % des nœuds sont en dehors de la composante géante, avec :

$$\alpha = e^{-c(1-\alpha)}$$

Théorème 2 (de Menger). *Soient G un graphe non-orienté, et x et y deux sommets distincts. Le nombre maximal de chemins x-y internement disjoints est égal au nombre minimal de sommets dont la suppression sépare x de y.*

Dans le contexte du réseau P2P, le nombre maximal de chemins disjoints entre le pair émetteur x et le pair récepteur y est égal à la résilience minimale de la connexion (x, y) contre la défaillance des intermédiaires.

Le théorème 1 nous donne le seuil pour la 1-connexité (connexité simple, où $k = 1$) du graphe total. Pour assurer une résilience, nous avons besoin d'une k-connexité avec $k \geq 2$. Le seuil de probabilité $p(n)$ requis pour qu'un graphe d'Erds-Rényi soit k-connexe est :

Théorème 3 (Seuil de k-Connexité). *Pour tout entier fixe $k \geq 1$, la probabilité d'arête $p(n)$ tel que*

$$p(n) \geq \frac{\log n + (k-1)\log(\log n) + \omega(1)}{n}$$

est un seuil critique pour la k-connexité de $G(n, p(n))$.

Partons des postulats que les serveurs d'un réseau (m) centralisé soient entièrement connectés ($p_m = 1$) et que m croît plus lentement que n [1]. On cherche donc quand un réseau pair-à-pair est plus résilient qu'un réseau centralisé, sachant que $n > m$, c'est-à-dire que dans les deux cas, il y a plus d'utilisateurs que de serveurs – ce qui est le cas empiriquement.

Notons $\mathcal{K}$ la fonction associé au théorème 2. Nous cherchons donc lorsque :

$$\mathcal{K}(G_{p2p}) \geq \mathcal{K}(G_m)$$

Ainsi, pour déconnecter deux nœuds dans $\mathcal{K}_m$, il faut retirer les $m-2$ autres nœuds, donc la connectivité de $\mathcal{K}_m$ est :

$$\mathcal{K}(G_{\text{serveur}}) = \mathcal{K}(G(m,1)) = m-1 \tag{1}$$

Donc, pour que le réseau pair-à-pair soit au moins résilient que le réseau centralisé, il faut que :

$$\mathcal{K}(G_{p2p}) \geq m-1$$

Injectons (1) dans la condition obtenue au théorème 3.

$$p(n) \gtrsim \frac{\log n + ((m(n)-1)-1)\log(\log n) + \omega(1)}{n} = \boxed{\frac{\log n + (m(n)-2)\log(\log n) + \omega(1)}{n}}$$

1. Par exemple, on peut avoir $m = \Theta(\frac{\log n}{\log\log n})$. Empiriquement, on observe que m croît plus lentement que n. Ce postulat permet de garantir une solution pour un n très grand.

Conclusion. Bien que le réseau pair-à-pair ait plus de nœuds, il n'est pas plus résilient par nature. Il ne le devient que lorsque sa densité de connexion $p(n)$ est suffisamment élevée pour atteindre le seuil de m-connexité, garantissant au moins m chemins disjoints entre les pairs, surpassant ainsi les $m-1$ chemins disjoints du réseau centralisé $\mathcal{K}_m$.

Critique du modèle. Nous nous sommes néanmoins intéressé à un cas très favorable au réseau centralisé, représenté sous forme d'un graphe complet. Dans la pratique, tous les serveurs ne sont pas connectés entre eux, donc $p_m < 1$. Le réseau pair-à-pair est donc résilient plus rapidement que le réseau centralisé. Néanmoins, nous omettons la différence entre suppression aléatoire et suppression stratégique. Dans ce dernier cas, les nœuds les plus connectés sont supprimés et la résilience du réseau pair-à-pair est plus difficile.

Cas d'un réseau point-à-point. Dans le cas d'un réseau point-à-point, lorsque les deux pairs ($n = 2$) sont connectés, alors $p = 1$. Les deux pairs forment, par nature, une composante géante, avec $\alpha = 0$ et un seul chemin. Ce qui implique nécessairement qu'il y a un point de défaillance unique. La démonstration résultante est donc moins intéressante. Mais cette défaillance se produit uniquement pour ces deux pairs et non pour le réseau entier : le reste des connexions continent de fonctionner.

Références

[1] Léo GAYRAL. *Graphes Aléatoires (Notes de cours)*. Page personnelle de Léo Gayral (CNRS). Consulté le 26/10/2025. 2017. URL : https://lgayral.pages.math.cnrs.fr/misc/graphes2017.pdf.

[2] Paul THEVENIN. *Graphes d'Erdös-Rényi et grandes déviations Introduction au domaine de recherche*. Consulté le 26/10/2025. Oct. 2016. URL : https://www.math.ens.psl.eu/shared-files/9731/?IDR_THEVENIN.pdf.

[3] ERDÖS, PAUL et RÉNYI, ALFRÉD. « On the evolution of random graphs ». In : *Publications of the Mathematical Institute of the Hungarian Academy of Sciences* 5 (1960), p. 17-61.